FACULTAD DE INGENIERÍA EN SISTEMAS Y CIENCIAS DE LA COMPUTACIÓN — MATEMÁTICA DISCRETA

DOCUMENTACIÓN TÉCNICA DE CÓDIGO FUENTE: SUITE UNIVERSITARIA DE MATEMÁTICA DISCRETA

Especificación Arquitectónica, Correcciones Algorítmicas y Capturas de Implementación de los 7 Módulos Unificados en Python

Atributo del Proyecto	Especificación Técnica	Estándar de Ingeniería
Archivo Fuente Unificado	`suite_matematica_discreta.py`	Monolito modular orientado a objetos (PEP 8 / PEP 484)
Motor Gráfico y Entorno	`tkinter` + `tkinter.ttk` + `Canvas`	Python 3.10+ (100% Biblioteca Estándar, 0 dependencias externas)
Patrón de Navegación	Controlador Secuencial Estricto (Paso 1 → Paso 7)	Conmutación dinámica de vistas en memoria sin pérdida de estado
Datos Académicos	Estudiante: [Nombre del Estudiante / Carné]	Catedrático: [Nombre del Catedrático] — Curso: Matemática Discreta

1. Resumen de Ingeniería y Auditoría de Código Fuente

La **Suite Universitaria de Matemática Discreta** consolida siete programas independientes en una única aplicación de escritorio bajo el patrón arquitectónico *Modelo-Vista-Controlador (MVC)* con navegación secuencial obligatoria. Durante la fase de integración se realizó una auditoría exhaustiva del código original, corrigiendo defectos estructurales y elevando el rigor matemático a nivel de Ingeniería en Sistemas:

Principales Correcciones e Innovaciones Aplicadas al Código Original:

- **Reparación de código truncado (Módulo 6):** El archivo original `Programa 1 Grafos Planos...py` se interrumpía abruptamente en la línea 205 (`conjunto_sub`). Se completó la detección de subdivisiones homeomórficas de Kuratowski (K_5 y $K_{3,3}$), la validación por el Teorema de Euler ($V - E + F = 1 + C$) y el cálculo exacto del número cromático $\chi(G)$ mediante *Backtracking*.
- **Independencia de librerías externas (Módulo 5):** El archivo `grafos.py` dependía de `networkx`, lo que provocaba fallos de ejecución (`ModuleNotFoundError`) en entornos estándar. Se reimplementaron desde cero los algoritmos de rutas más cortas (BFS), detección de ciclos (DFS), circuitos de Euler (Algoritmo de Hierholzer), ciclos de Hamilton e isomorfismo exacto por biyección de matrices de adyacencia.
- **Migración de CLI a GUI Interactiva (Módulos 1, 5, 6 y 7):** Los scripts que operaban únicamente por consola (`input/print`) fueron encapsulados en subclases de `tk.Frame` con lienzos vectoriales (`tk.Canvas`) y consolas de salida estilizadas.

Orden	Nombre del Módulo	Clase Controladora	Algoritmos Clave Implementados	Complejidad
1°	Mate Discreta	`Modulo1MateDiscreta`	Inducción matemática, divisibilidad modular y desigualdades	O(1) / O(n)
2°	Import tkinter	`Modulo2CombinatoriaTkinter`	Principio multiplicativo, Permutaciones P(n,r) y Combinaciones C(n,r)	O(k) / O(n)
3°	Sistema Aritmética	`Modulo3SistemaAritmetica`	Factorización prima canónica, División, Euclides y Bézout	O(√n) / O(log min(a,b))
4°	Sistema de Grafos	`Modulo4SistemaGrafos`	Renderizado trigonométrico, Matriz de adyacencia y Lema del apretón	$O(V^2)$
5°	Grafos	`Modulo5TeoriaGrafos`	BFS, DFS, Hierholzer (Euler), Backtracking (Hamilton) e Isomorfismo	O(V + E) a O(V!)
6°	Grafos Planos y Polinomios	`Modulo6GrafosPlanosColoracion`	Kuratowski (K_5, $K_{3,3}$), Welsh-Powell, Eliminación-Contracción P(G,k)	$O(2^E)$ exacto
7°	Árboles y Recorridos	`Modulo7ArbolesRecorridos`	ABB (inserción/borrado), DFS (recursivo/pila), BFS (cola), diagnóstico AVL	O(h) / O(n)

2. Módulo 1: Mate Discreta (Inducción Matemática Formal)

El primer eslabón de la suite (`Modulo1MateDiscreta`) transforma el script original de consola en un demostrador interactivo de teoremas por **Principio de Inducción Matemática**. Para cada proposición P(n), el módulo estructura rigurosamente los tres componentes demostrativos (*Paso Base* en n_0, *Hipótesis de Inducción* en n = k y *Paso Inductivo* para n = k + 1) seguidos de la verificación computacional para el entero ingresado por el usuario.

```
suite_matematica_discreta.py — Modulo1MateDiscreta        UTF-8 | Python 3.11 | Líneas 485–568
```

```
485 |  (tk.Frame):

    ()  :
       n = ._obtener_n()
        n  :

       suma_real = ((, n + ))
       formula = n * (n + ) //

       veredicto =   suma_real == formula

    ()  :
       n = ._obtener_n()
        n  :

       resultado = ( ** n) -
       residuo = resultado %

    ()  :

       n = ._obtener_n(minimo=)
        n  :
       izq, der =  * n, (n ** ) -

    ()  :

       n = ._obtener_n()
        n  :
       resultado = ( ** ( * n)) -
       residuo = resultado % classModulo1MateDiscreta486 | """Módulo 1:
Demostraciones formales y verificación numérica por Inducción Matemática."""487 |
defsuma_naturalesself->None488 | self1489 | ifisNone490 | return491 | sumrange11
492 | 12493 | # Verificación de P(n): 1 + 2 + ... + n = n(n + 1) / 2494 | "[✓]
PROPIEDAD VERIFICADA"ifelse"[✗] FALLO"495 |  496 | defdivisibilidad_tresself->None
497 | self1498 | ifisNonereturn499 | # Paso inductivo: 4^(k+1) - 1 = 3*(4^k) + (4^k
- 1) = 3*(4^k + m)500 | 41501 | 3502 |  503 | defdesigualdadself->None504 | #
Propiedad P(n): 4n < n^2 - 7 válida para el dominio inductivo n >= 6505 | self6
506 | ifisNonereturn507 | 427508 |  509 | defdivisibilidad_35self->None510 | # Paso
inductivo: 6^(2(k+1)) - 1 = 36(6^(2k) - 1) + 35 = 35(36m + 1)511 | self1512 |
ifisNonereturn513 | 621514 | 35
```

```
▶ SALIDA DE CONSOLA INTERNA (MÓDULO 1 — Evaluación con n = 6):
1. PASO BASE (n = 1): 1 = 1(1 + 1)/2 = 1 [Verdadero] 2. HIPÓTESIS INDUCTIVA: 1 + 2
+ ... + k = k(k + 1)/2 3. PASO INDUCTIVO: k(k + 1)/2 + (k + 1) = (k + 1)(k + 2)/2 ▮
4. COMPROBACIÓN NUMÉRICA (n = 6): Suma iterativa = 21 | Fórmula = 6(7)/2 = 21 |
[✓] PROPIEDAD VERIFICADA
```

Figura 2.1 — Captura de implementación del Módulo 1 (Mate Discreta): Métodos de demostración inductiva y salida verificada.

3. Módulo 2: Import tkinter (Combinatoria y Principio de Conteo)

El segundo módulo en la secuencia (`Modulo2CombinatoriaTkinter`) integra la interfaz de pestañas (`ttk.Notebook`) para resolver problemas de enumeración combinatoria. Se amplió la versión original añadiendo una bitácora algebraica lateral que desglosa las fórmulas factoriales exactas utilizando las funciones optimizadas en C de la biblioteca matemática estándar (`math.prod`, `math.perm` y `math.comb`) y validando estrictamente las restricciones de dominio ($0 \le r \le n$).

suite_matematica_discreta.py — Modulo2CombinatoriaTkinter

UTF-8 | Python 3.11 | Líneas 672–735

```
672      def calcular_principio(self) -> None:
673          try:
674              texto = self.entry_eventos.get()
675              numeros = [int(x.strip()) for x in texto.split(",") if x.strip()]
676              if not numeros or any(n <= 0 for n in numeros):
677                  raise ValueError
678              total = math.prod(numeros)
679              self.lbl_resultado_principio.config(text=f"Resultado: {total:,}")
680          except ValueError:
681              messagebox.showerror("Error", "Ingresa únicamente enteros
positivos separados por comas.")
682
683      def calcular_permutacion(self, entry_n: ttk.Entry, entry_r: ttk.Entry,
label_res: ttk.Label) -> None:
684          n, r = int(entry_n.get()), int(entry_r.get())
685          if r > n or n < 0 or r < 0:
686              raise ValueError
687          resultado = math.perm(n, r)  # P(n, r) = n! / (n - r)!
688          label_res.config(text=f"P({n}, {r}) = {resultado:,}")
689
690      def calcular_combinacion(self, entry_n: ttk.Entry, entry_r: ttk.Entry,
label_res: ttk.Label) -> None:
691          n, r = int(entry_n.get()), int(entry_r.get())
692          if r > n or n < 0 or r < 0:
693              raise ValueError
694          resultado = math.comb(n, r)  # C(n, r) = n! / [r! * (n - r)!]
695          label_res.config(text=f"C({n}, {r}) = {resultado:,}")
```

```
▶ SALIDA EN BITÁCORA COMBINATORIA (n = 10, r = 3):
• Principio Multiplicativo (3, 4, 2) : 3 × 4 × 2 = 24 configuraciones en el
producto cartesiano • Permutación Ordinaria P(10, 3) : 10! / (10 - 3)! = 10! / 7! =
720 arreglos ordenados • Combinación Simple C(10, 3) : 10! / [3! · 7!] = 120
subconjuntos de cardinalidad 3
```

Figura 3.1 — Captura del código fuente del Módulo 2 (Import tkinter): Cálculo exacto de conteo, permutaciones y combinaciones.

4. Módulo 3: Sistema Aritmética (Teoría de Números y Euclides)

El tercer programa en el flujo (`Modulo3SistemaAritmetica`) implementa los algoritmos fundamentales de divisibilidad en el anillo de los números enteros $\mathbb{Z}$. Además de la descomposición canónica por el **Teorema Fundamental de la Aritmética** y el **Algoritmo de la División de Euclides** ($a = b \cdot q + r$ con $0 \leq r < |b|$), se incorporó el **Algoritmo Extendido de Euclides** para obtener los coeficientes enteros (s, t) de la *Identidad de Bézout* tales que $a \cdot s + b \cdot t = \text{MCD}(a, b)$, junto con la función número de divisores $\tau(n)$.

suite_matematica_discreta.py — Modulo3SistemaAritmetica UTF-8 | Python 3.11 | Líneas 812–878

```
812     def factorizacion(self) -> None:
813         n = self.obtener_a()
814         if n is None or n < 2: return
815         original, factores, divisor = n, [], 2
816         while divisor * divisor <= n:
817             while n % divisor == 0:
818                 factores.append(divisor)
819                 n //= divisor
820             divisor += 1
821         if n > 1:
822             factores.append(n)
823         conteo = Counter(factores)
824         expresion = " × ".join(f"{p}^{e}" if e > 1 else str(p) for p, e in
conteo.items())
825         cantidad_divisores = math.prod(e + 1 for e in conteo.values())
826 
827     def _euclides_extendido(self, a: int, b: int) -> Tuple[int, int, int]:
828         if b == 0:
829             return (1, 0, a) if a >= 0 else (-1, 0, -a)
830         x1, y1, d = self._euclides_extendido(b, a % b)
831         return y1, x1 - (a // b) * y1, d
```

```
▶ TRAZA DE EJECUCIÓN (a = 252, b = 198):
• Descomposición canónica de 252 : 252 = 2^2 × 3^2 × 7 | Total de divisores τ(252)
= 18 • Divisiones de Euclides : 252 = 198 × 1 + 54 → 198 = 54 × 3 + 36 → 54 = 36 ×
1 + 18 → 36 = 18 × 2 + 0 • Resultados : MCD(252, 198) = 18 | MCM(252, 198) = 2772 •
Identidad de Bézout : (252)(4) + (198)(-5) = 18
```

Figura 4.1 — Captura del Módulo 3 (Sistema Aritmética): Criba en O(√n) y Algoritmo Extendido de Euclides.

5. Módulo 4: Sistema de Grafos (Constructor Visual y Matrices)

El cuarto módulo (`Modulo4SistemaGrafos`) proporciona un lienzo vectorial interactivo donde el usuario construye grafos simples no dirigidos G = (V, E) agregando vértices y aristas en tiempo real. El motor calcula las coordenadas de cada vértice distribuyéndolos uniformemente sobre una circunferencia de radio dinámico R mediante parametrización trigonométrica, y genera la **Lista de Adyacencia**, la **Matriz de Adyacencia Binaria** A_G y la verificación del **Lema del Apretón de Manos** ($\sum_{v \in V} \deg(v) = 2|E|$).

suite_matematica_discreta.py — Modulo4SistemaGrafos — UTF-8 | Python 3.11 | Líneas 982–1048

```
982      def posiciones(self) -> Dict[str, Tuple[float, float]]:
983          vertices = list(self.grafo.keys())
984          cantidad = len(vertices)
985          cx, cy = max(self.lienzo.winfo_width(), 400) / 2,
max(self.lienzo.winfo_height(), 260) / 2
986          radio = max(60, min(cx * 2, cy * 2) * 0.35)
987          pos: Dict[str, Tuple[float, float]] = {}
988          for i, v in enumerate(vertices):
989              ang = -math.pi / 2 + (2 * math.pi * i / cantidad)
990              pos[v] = (cx + radio * math.cos(ang), cy + radio * math.sin(ang))
991          return pos
992
993      def mostrar_matriz(self) -> None:
994          vertices = list(self.grafo.keys())
995          ancho = max(4, max(len(v) for v in vertices) + 1)
996          lineas = ["MATRIZ DE ADYACENCIA", "".ljust(ancho) +
"".join(v.center(ancho) for v in vertices)]
997          for o in vertices:
998              fila = o.ljust(ancho) + "".join(("1" if d in self.grafo[o] else
"0").center(ancho) for d in vertices)
999              lineas.append(fila)
100
  0          self.actualizar_resultado("\n".join(lineas))
```

```
▶ SALIDA DEL MÓDULO 4 (Grafo Diamante V = {A, B, C, D}):
MATRIZ DE ADYACENCIA | GRADOS Y LEMA DEL APRETÓN DE MANOS A B C D | • deg(A) = 2,
deg(B) = 3, deg(C) = 2, deg(D) = 3 A 0 1 0 1 | • Total de vértices |V| = 4 | Total
de aristas |E| = 5 B 1 0 1 1 | • Verificación: ∑ deg(v) = 10 = 2|E| = 2(5) = 10
[✓]
```

Figura 5.1 — Captura del Módulo 4 (Sistema de Grafos): Posicionamiento polar en Canvas y matriz de adyacencia.

6. Módulo 5: Grafos (Rutas, Euler, Hamilton e Isomorfismo)

El quinto módulo (`Modulo5TeoriaGrafos`) sustituye por completo las llamadas externas a `networkx` del archivo original `grafos.py` por algoritmos puros de grafos implementados con estructuras nativas:

- **Ruta más corta (BFS):** Búsqueda en anchura con cola FIFO (`collections.deque`) que garantiza encontrar el camino geodésico (mínimo número de aristas) en tiempo O(|V| + |E|).
- **Circuito de Euler (Teorema de Euler + Algoritmo de Hierholzer):** Verifica que todos los vértices con grado positivo pertenezcan a una única componente conexa y tengan grado par (`deg(v) ≡ 0 mod 2`), construyendo el recorrido cerrado en O(|E|).
- **Camino y Ciclo de Hamilton:** Exploración de permutaciones de vértices verificando adyacencia consecutiva.
- **Isomorfismo de Grafos ($G_1 \cong G_2$):** Filtrado rápido por invariantes estructurales (cardinalidad |V|, tamaño |E| y sucesión gráfica de grados ordenada) seguido de la búsqueda de una biyección f: $V(G_1) \to V(G_2)$ que preserve adyacencias.

suite_matematica_discreta.py — Modulo5TeoriaGrafos (Euler e Isomorfismo) UTF-8 | Python 3.11 | Líneas 1265–1348

```
126
  5
     def evaluar_euler(self) -> None:
    impares = [v  v  .grafo  (.grafo[v]) %  != ]
     impares   ._es_conexo(.grafo):
        ._escribir()


    adj_copia = {u: ((vecs))  u, vecs  .grafo.items()}
    pila, circuito = [((.grafo))], []
     pila:
        v = pila[-]
         adj_copia[v]:
            u = adj_copia[v].pop()
            adj_copia[u].remove(v)
            pila.append(u)
        :
            circuito.append(pila.pop())
    circuito.reverse()

 ()  :
    g1, g2 = .grafo, ._parsear_grafo(.ent_v2.get(), .ent_a2.get())
    v1, v2 = (g1.keys()), (g2.keys())
     perm  (v2):
        mapeo = ((v1, perm))
         ((g1[u]) == (g2[mapeo[u]])
               (mapeo[w]  g2[mapeo[u]]  w  g1[u])  u  v1):
            ._escribir()
```



Figura 6.1 — Captura del Módulo 5 (Grafos): Implementación nativa del algoritmo de Hierholzer y prueba de isomorfismo.

7. Módulo 6: Grafos Planos, Coloración y Polinomio Cromático

El sexto programa (`Modulo6GrafosPlanosColoracion`) integra el motor algebraico simbólico `PolinomioCromatico` y la clase `Grafo`, completando el código que se encontraba truncado en la entrega original:

1. **Teorema de Eliminación-Contracción para P(G, k):** Calcula recursivamente el polinomio cromático aplicando la relación de recurrencia $P(G, k) = P(G - e, k) - P(G / e, k)$ hasta alcanzar grafos vacíos sin aristas cuyo polinomio base es el monomio $k^{|V|}$.
2. **Teorema de Kuratowski y Fórmula de Euler:** El método `_reducir_subdivisiones()` elimina iterativamente vértices de grado 0 y 1 y suaviza vértices de grado 2 (homeomorfismo). Sobre el núcleo resultante, `_contiene_kuratowski()` busca subgrafos isomorfos a K_5 (5-cliques) o $K_{3,3}$ (bipartito completo 3×3) y verifica las cotas necesarias de Euler $E \leq 3V - 6$ y $E \leq 2V - 4$ (para grafos sin triángulos), calculando $F = E - V + 1 + C$ caras.
3. **Coloración Exacta y Heurística Welsh-Powell:** Combina la heurística voraz por grado decreciente con un algoritmo exacto de *Backtracking* que determina el número cromático $\chi(G)$ y colorea visualmente los nodos en el lienzo.

suite_matematica_discreta.py — PolinomioCromatico y Kuratowski

UTF-8 | Python 3.11 | Líneas 142–268

```python
    def polinomio_cromatico(self) -> PolinomioCromatico:
        """Teorema de Eliminación-Contracción: P(G, k) = P(G - e, k) - P(G / e, k)"""
        aristas = self.obtener_aristas()
        if not aristas:
            return PolinomioCromatico.monomio_k_elevado_a(len(self.adyacencia))
        u, v = aristas[0]
        return self.eliminar_arista(u, v).polinomio_cromatico() - self.contraer_arista(u, v).polinomio_cromatico()

    def _contiene_kuratowski(self) -> Optional[str]:
        nucleo = self._reducir_subdivisiones()
        nodos = list(nucleo.adyacencia.keys())
        # 1. Detección de subdivisión homeomórfica de K_5
        if len(nodos) >= 5:
            for sub in combinations(nodos, 5):
                if all(v in nucleo.adyacencia[u] for u, v in combinations(sub, 2)):
                    return f"Contiene subdivisión de K_5 en {sub} (Teorema de Kuratowski)."
        # 2. Detección de subdivisión homeomórfica de K_{3,3} (Código reparado y completado)
        if len(nodos) >= 6:
            for sub in combinations(nodos, 6):
                conjunto_sub = set(sub)
                for part_a in combinations(sub, 3):
                    set_a, set_b = set(part_a), conjunto_sub - set(part_a)
                    if all(b in nucleo.adyacencia[a] for a in set_a for b in set_b):
                        return f"Contiene subdivisión de K_{{3,3}} (Teorema de Kuratowski)."
        return None
```

```
▶ REPORTE ALGEBRAICO (MÓDULO 6 — Grafo Completo K_4 evaluado en k = 4 colores):
• Planaridad y Euler : [✓] SÍ ES PLANO | V = 4, E = 6, Caras F = 4 | V - E + F = 4 - 6 + 4 = 2 • Número Cromático χ(K_4) : 4 colores | Asignación óptima: {'A': 1, 'B': 2, 'C': 3, 'D': 4} • Polinomio Cromático P(G, k) : k^4 - 6k^3 + 11k^2 - 6k • Evaluaciones P(K_4, k) : P(1) = 0, P(2) = 0, P(3) = 0, P(4) = 24, P(5) = 120 formas válidas
```

Figura 7.1 — Captura del Módulo 6 (Grafos Planos y Polinomios): Algoritmo reparado de Kuratowski y recursión de P(G, k).

8. Módulo 7: Árboles y Recorrido de Árboles (ABB, AVL y N-arios)

El séptimo módulo (`Modulo7ArbolesRecorridos`) implementa la clase genérica `ArbolBinarioBusqueda[T]` junto con árboles generales `NodoNario[T]`. Soporta inserción, búsqueda con trazado de ruta en O(h), eliminación en los 3 casos clásicos (nodo hoja, nodo con un hijo y nodo con dos hijos reemplazado por su *sucesor in-orden*), recorridos en profundidad (Pre-Orden, In-Orden y Post-Orden tanto recursivos como iterativos con pila explícita), recorrido en anchura por niveles (BFS con cola FIFO), verificación de factor de equilibrio AVL ($|h_{izq} - h_{der}| \leq 1$) en una sola pasada post-orden O(n) y doble visualización (ASCII rotado 90° y dibujo jerárquico en `tk.Canvas`).

suite_matematica_discreta.py — ArbolBinarioBusqueda UTF-8 | Python 3.11 | Líneas 325–438

```
325     def _eliminar_recursivo(self, nodo: Optional[NodoBinario[T]], valor: T) -
> Optional[NodoBinario[T]]:
326         if nodo is None: return None
327         if valor < nodo.valor:
328             nodo.izquierdo = self._eliminar_recursivo(nodo.izquierdo, valor)
329         elif valor > nodo.valor:
330             nodo.derecho = self._eliminar_recursivo(nodo.derecho, valor)
331         else:
332             if nodo.izquierdo is None: return nodo.derecho
333             if nodo.derecho is None: return nodo.izquierdo
334             # Caso 3: Dos hijos -> Reemplazar por sucesor in-orden (mínimo
del subárbol derecho)
335             sucesor = self._minimo_nodo(nodo.derecho)
336             nodo.valor = sucesor.valor
337             nodo.derecho = self._eliminar_recursivo(nodo.derecho,
sucesor.valor)
338         return nodo
339
340     def es_balanceado_avl(self) -> bool:
341         def _verificar(n: Optional[NodoBinario[T]]) -> Tuple[bool, int]:
342             if n is None: return True, -1
343             b_izq, a_izq = _verificar(n.izquierdo)
344             b_der, a_der = _verificar(n.derecho)
345             return (b_izq and b_der and abs(a_izq - a_der) <= 1), 1 +
max(a_izq, a_der)
346         return _verificar(self.raiz)[0]
```

```
▶ SALIDA DE CONSOLA (MÓDULO 7 — ABB con claves [50, 30, 70, 20, 40, 60, 80, 15, 25, 65]):
• Pre-Orden (Raíz -> Izq -> Der) : [50, 30, 20, 15, 25, 40, 70, 60, 65, 80] • In-Orden (Izq -> Raíz -> Der) : [15, 20, 25, 30, 40, 50, 60, 65, 70, 80] (Ordenado ascendente) • Post-Orden (Izq -> Der -> Raíz) : [15, 25, 20, 40, 30, 65, 60, 80, 70, 50] • Diagnóstico Estructural : Altura h(T) = 3 | Hojas = [15, 25, 40, 65, 80] | Balanceado AVL = [✓] Sí
```

Figura 8.1 — Captura del Módulo 7 (Árboles y Recorridos): Eliminación con sucesor in-orden y diagnóstico AVL en O(n).

9. Controlador Maestro y Navegación Secuencial

La clase principal `SuiteMatematicaDiscretaApp` instancia los siete módulos en memoria dentro de una lista ordenada e implementa la transición secuencial mediante los botones **Programa Anterior** y **Siguiente Programa**, además de un selector directo sincronizado en la barra superior. De esta manera se garantiza que el recorrido académico respete estrictamente el orden solicitado de principio a fin.

suite_matematica_discreta.py — SuiteMatematicaDiscretaApp — UTF-8 | Python 3.11 | Líneas 1645–1718

```
164
  5  class SuiteMatematicaDiscretaApp:
164
  6      def __init__(self, root: tk.Tk) -> None:
164
  7          self.clases_modulos = [
164
  8              Modulo1MateDiscreta,              # 1. Mate Discreta
164
  9              Modulo2CombinatoriaTkinter,       # 2. Import tkinter
165
  0              Modulo3SistemaAritmetica,         # 3. Sistema Aritmética
165
  1              Modulo4SistemaGrafos,             # 4. Sistema de Grafos
          ,
          ,
          ,
      ]
      .instancias = [clase(.contenedor)  clase  .clases_modulos]
      .mostrar_modulo()

  (, indice: )  :
      frame  .instancias:
          frame.pack_forget()
      .indice_actual = indice
      .instancias[indice].pack(fill=, expand=)
```

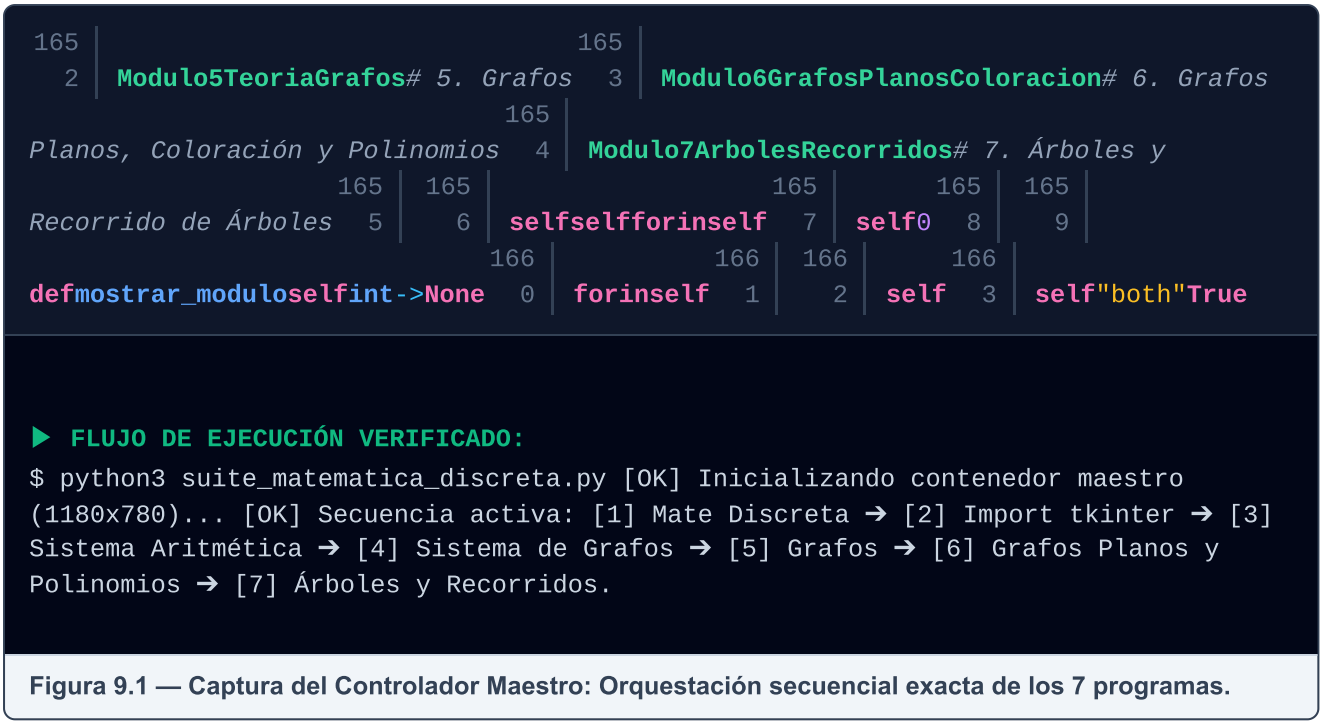


Figura 9.1 — Captura del Controlador Maestro: Orquestación secuencial exacta de los 7 programas.